Wrangling III homework

Deleting invalid rows from the cancer data

It's been decided by committee that duplicate data in the (smaller version of) the cancer data set is going to go as follows. If a row is identical to the one immediately above it, we'll consider it an accidental entry due to fatigue or whatever. But a row that is *not* identical to the one above it will be considered valid, even it has a duplicate somewhere else in the data set; we'll assume such duplicates represent separate visits.

So our rule is: **if a row is identical to the one above it, we drop it**.

A small pre-cleaned version of the data with only 4 columns is in **small_cancer_data.csv**, so you can read it in directly without having to clean it up.

Spend a minute or two thinking about how you would approach this problem.

If you are ready to go on your own, then go!

Once you have working code – once you can take **small_cancer_data.csv** and trim out the unwanted rows – then wrap your code into a function, so all you have to do drop unwanted rows is call your function!

Spend some time thinking about and working on the problem. If you get to an impass and you'd like some hints, read on.

Preliminaries

As usual, we'll load some libraries we'll be likely to use.

In [1]:
```
import pandas as pd
```

Make a mini data set for testing

Rather than taking a crack at the whole set, make a small data frame named `tiny` with 10 rows and two columns. Put successive repeated rows in two places (like rows 2 and 3

could repeat, as could rows 6 and 7). Put an additional repeated row on it's own.

Something like this:

In [2]:
```
tiny = pd.DataFrame(dict(a = [1, 2, 3, 3, 4, 5, 5, 5, 6, 3],
                         b = ['a', 'b', 'c', 'c', 'd', 'e', 'e', 'f', 'g', '
```

Check our tiny data frame.

In [3]:
```
# check your test data frame
tiny[tiny.duplicated()]
```

Out[3]:

	a	b
3	3	c
6	5	e
9	3	c

There should be two rows that need to be dropped, and one (the last) that should be kept even though it's a duplicate.

Just to be sure, check the output of `.duplicated(keep=False)` – it should show back-to-back `True` values in 2 places, and one solo `True` at the end.

In [4]:
```
# check .duplicated output
tiny.duplicated(keep=False)
```

Out[4]:
```
0    False
1    False
2     True
3     True
4    False
5     True
6     True
7    False
8    False
9     True
dtype: bool
```

Make a plan

There are probably 100 ways to solve this problem. Many are probably very clever and involve using fancy pandas functions.

A straightforward plan using things we already know about might be something like

- go through the rows of the data frame with a `for` loop, starting with the second row
- at each row, compare the current row with the previous one
- if they're the same, save the index of the current row
- after the `for` loop, delete the unwanted rows using the saved indexes

---

Test the parts of the plan

Now that we've got a plan, let's get the pieces of the plan to work before putting the whole plan together.

Make sure we can get rows

We should be able to get rows of a data frame in a couple of ways. These are

- using `.loc[]` with the value of rows index (it's name)
- using `.iloc[]` and indexing into the data like it were a numpy array

Let's try the `.loc[]` way.

In [6]:
```
df = pd.read_csv("./data/small_cancer_data.csv")

rows_to_drop = []
index_list = list(df.index)

for i in range(1, len(index_list)):
    current_index = index_list[i]
    previous_index = index_list[i - 1]

    if df.loc[current_index].equals(df.loc[previous_index]):
        rows_to_drop.append(current_index)

df_cleaned_loc = df.drop(rows_to_drop)

df_cleaned_loc
```

Out[6]:

	id	thick	chrom	class
0	1000025	5.0	3.0	benign
1	1002945	5.0	3.0	benign
2	1015425	3.0	3.0	benign
3	1016277	6.0	3.0	benign
4	1017023	4.0	3.0	benign
...	...	...	...	...
693	763235	3.0	2.0	benign
694	776715	3.0	1.0	benign
695	841769	2.0	1.0	benign
696	888820	5.0	8.0	malignant
697	897471	4.0	10.0	malignant

692 rows × 4 columns

And let's try the `.iloc[]` method.

In [7]:

```
rows_to_drop = []

for i in range(1, len(df)):
    if df.iloc[i].equals(df.iloc[i - 1]):
        rows_to_drop.append(df.index[i])

df_cleaned_iloc = df.drop(rows_to_drop)

df_cleaned_iloc
```

Out[7]:

	id	thick	chrom	class
0	1000025	5.0	3.0	benign
1	1002945	5.0	3.0	benign
2	1015425	3.0	3.0	benign
3	1016277	6.0	3.0	benign
4	1017023	4.0	3.0	benign
...	...	...	...	...
693	763235	3.0	2.0	benign
694	776715	3.0	1.0	benign
695	841769	2.0	1.0	benign
696	888820	5.0	8.0	malignant
697	897471	4.0	10.0	malignant

692 rows × 4 columns

Look's like either will work!

Figure out how to compare rows

We are going to need to compare rows. Let's see how that is going to work.

compare the first and second rows - these *should* *__not__* match

In [ ]:
```
# which things in the rows match?
tiny.loc[0].equals(tiny.loc[1])
```

Out[ ]:
```
False
```

compare the third and fourth rows - these *should* match

In [11]:
```
# which things in the rows match?
tiny.loc[2].equals(tiny.loc[3])
```

Out[11]:
```
True
```

The rows only match if *__all__* the columns match, so we can see if this is the case with the `all()` function.

In [12]:
```
# do all the columns match?
(tiny.iloc[2] == tiny.iloc[3]).all()
```

Out[12]:
```
True
```

Now we have a way to compare rows and get a single `True` if the rows are identical, and a `False` if they're not.

And now that we know how to do the row comparison, let's get a `for` loop working.

Confirm we can get rows with a `for` loop

Loop through the first few rows

Let's make sure we can index into rows with a for loop. Let's try to get the first few using `.loc[]` and print them. Like

```
for ... :
    print(...)
```

In [13]:
```
# loop through the first few rows
for idx in tiny.index[:5]:
    print(tiny.loc[idx])
    print("-----")
```

```
a    1
b    a
Name: 0, dtype: object
-----
a    2
b    b
Name: 1, dtype: object
-----
a    3
b    c
Name: 2, dtype: object
-----
a    3
b    c
Name: 3, dtype: object
-----
a    4
b    d
Name: 4, dtype: object
-----
```

*Loop through the all** rows*

To loop through all the rows, we first need to get the number of rows. We can do this using the `shape` attribute.

In [14]:
```
# get the number of rows using shape
n_rows = tiny.shape[0]
print(n_rows)
```

```
10
```

In [15]:
```
# loop through all the rows
for i in range(n_rows):
    print(tiny.loc[i])
    print("-----")
```

```
a    1
b    a
Name: 0, dtype: object
-----
a    2
b    b
Name: 1, dtype: object
-----
a    3
b    c
Name: 2, dtype: object
-----
a    3
b    c
Name: 3, dtype: object
-----
a    4
b    d
Name: 4, dtype: object
-----
a    5
b    e
Name: 5, dtype: object
-----
a    5
b    e
Name: 6, dtype: object
-----
a    5
b    f
Name: 7, dtype: object
-----
a    6
b    g
Name: 8, dtype: object
-----
a    3
b    c
Name: 9, dtype: object
-----
```

Putting it all together

Get the number of rows

In [16]:
```
# get the number of rows using shape
n_rows = tiny.shape[0]
```

Make an empty list to hold the indexes of the rows we're going to drop

In [17]:
```
rows_to_drop = []
```

Make a `for` loop that

- goes from 1 (i.e. the second row) to the end
- tests the current row against previous
- stores index for dropping

In [18]:
```
for i in range(1, n_rows):
    if (tiny.loc[i] == tiny.loc[i-1]).all():
        rows_to_drop.append(i)

print(rows_to_drop)
```

```
[3, 6]
```

Check that we got the correct indexes.

In [19]:
```
for idx in rows_to_drop:
    print(f"Row {idx} (to drop):")
    print(tiny.loc[idx])
    print("Previous row:")
    print(tiny.loc[idx - 1])
    print("-----")
```

```
Row 3 (to drop):
a    3
b    c
Name: 3, dtype: object
Previous row:
a    3
b    c
Name: 2, dtype: object
-----
Row 6 (to drop):
a    5
b    e
Name: 6, dtype: object
Previous row:
a    5
b    e
Name: 5, dtype: object
-----
```

Make a new data frame with the unwanted rows `.drop`ped.

In [20]:
```
tiny_cleaned = tiny.drop(rows_to_drop)
```

```
tiny_cleaned
```

Out[20]:

	a	b
0	1	a
1	2	b
2	3	c
4	4	d
5	5	e
7	5	f
8	6	g
9	3	c

Use `.reset_index()` to make a new sequental index for our data frame.

In [21]:
```
tiny_cleaned = tiny_cleaned.reset_index(drop=True)
```

Marvel at your work!

In [22]:
```
tiny_cleaned
```

Out[22]:

	a	b
0	1	a
1	2	b
2	3	c
3	4	d
4	5	e
5	5	f
6	6	g
7	3	c

If you don't like the "index" column with old indexes (sometimes it's useful to have the old indexes – here it's just annoying), you can set `drop=True` when you call `.reset_index()` above.

Run your code on the cancer data

Try our code on the (small version of the) cancer data!

Load the data

In [24]:
```
cancer_df = pd.read_csv("./data/small_cancer_data.csv")

cancer_df
```

Out[24]:

	id	thick	chrom	class
0	1000025	5.0	3.0	benign
1	1002945	5.0	3.0	benign
2	1015425	3.0	3.0	benign
3	1016277	6.0	3.0	benign
4	1017023	4.0	3.0	benign
...	...	...	...	...
694	776715	3.0	1.0	benign
695	841769	2.0	1.0	benign
696	888820	5.0	8.0	malignant
697	897471	4.0	10.0	malignant
698	897471	4.0	10.0	malignant

699 rows × 4 columns

Get the number of rows

In [25]:
```
# get the number of rows using shape
n_rows = cancer_df.shape[0]
print("Number of rows:", n_rows)
```

```
Number of rows: 699
```

Make an empty list for indexes

In [26]:
```
rows_to_drop = []
```

Run your `for` loop!

In [27]:
```
for i in range(1, n_rows):
    # Compare current row to the previous row
    if (cancer_df.loc[i] == cancer_df.loc[i-1]).all():
        rows_to_drop.append(i)
```

Check the indexes you found

In [30]:
```
print(rows_to_drop)
```

```
[208, 322, 443, 561, 684, 690, 698]
```

Drop the unwanted rows

In [31]:
```
cancer_cleaned = cancer_df.drop(rows_to_drop)
```

Reset the row indexes

In [33]:
```
cancer_cleaned = cancer_cleaned.reset_index(drop=True)
```

Check the shape to confirm the rows were dropped!

In [34]:
```
print(cancer_cleaned.shape)
```

```
(692, 4)
```

Wrapping it all in a function

Once you've got your code running, put it all in a function so it's reusable!

In [35]:
```
def drop_consecutive_duplicates(df):
    n_rows = df.shape[0]
    rows_to_drop = []

    for i in range(1, n_rows):
        if (df.loc[i] == df.loc[i-1]).all():
            rows_to_drop.append(i)

    df_cleaned = df.drop(rows_to_drop)

    df_cleaned = df_cleaned.reset_index(drop=True)

    return df_cleaned
```

Run your function!

In [37]:
```
cancer_df = pd.read_csv("./data/small_cancer_data.csv")

cancer_cleaned = drop_consecutive_duplicates(cancer_df)
```

Check the shape to confirm your function worked!

In [38]:
```
print(cancer_cleaned.shape)
```

```
(692, 4)
```

High-five the person closest to you!

Because you deserve a high-five right now.